



**Academia de Ensino
Superior de Mafra**

Cursos Técnicos Superiores Profissionais (CTESP)



AEROGEST

Sistema de Gestão Simples de Aeródromo

AUTOR:

DIOGO SANTOS, N.º 82081

DISCIPLINA/PROFESSOR:

PROGRAMAÇÃO AVANÇADA / Fernando Barros

ACADEMIA DE ENSINO SUPERIOR DE MAFRA, 12 DE JULHO DE 2026

ÍNDICE

1 - Resumo	1
2 - Introdução.....	2
3 - Descrição do problema.....	3
4 - Desenvolvimento.....	4
4.1 - Estrutura da aplicação.....	4
4.2 - Menu principal.....	4
4.3 - Gestão de aeronaves	5
4.4 - Gestão de movimentos	6
4.5 - Filtragem de movimentos	7
4.6 - Validação e tratamento de erros	8
4.7 - Base de dados	9
4.8 - Geração do relatório HTML	10
5 - Resultados	11
5.1 - Resultados funcionais.....	11
5.2 - Testes realizados.....	12
5.3 - Problemas encontrados e resolução	12
6 - Conclusões.....	13
7 - Referências.....	14

1 - RESUMO

Neste trabalho desenvolvi uma aplicação de linha de comandos para a gestão básica de um aeródromo. A aplicação, com o nome AeroGest, foi criada em Python e utiliza uma base de dados MySQL para guardar aeronaves e movimentos de chegada e partida. O objetivo principal foi construir um programa simples, funcional e suficientemente claro para ser apresentado e explicado.

O sistema permite adicionar e listar aeronaves, alterar o seu estado, registar chegadas e partidas, consultar e filtrar movimentos, apresentar estatísticas e gerar um relatório em HTML. Foram também incluídas validações para impedir campos vazios, números inválidos, datas incorretas, opções inexistentes e matrículas repetidas.

Durante a preparação do projeto foram realizados testes funcionais e testes de erro. Os dados mantiveram-se guardados depois de fechar o programa, confirmando a ligação à base de dados. A versão final foi organizada em ficheiros separados para a interação com o utilizador, o acesso ao MySQL e a criação do relatório.

Palavras-chave: Python, MySQL, linha de comandos, aeródromo, validação, relatório HTML.

2 - INTRODUÇÃO

A gestão de um aeródromo envolve vários tipos de informação. Mesmo num sistema pequeno, é necessário saber que aeronaves estão registadas, qual é o seu estado e que movimentos estão previstos ou já foram realizados. Sem uma aplicação, estes dados podem ficar espalhados por ficheiros diferentes ou ser registados manualmente, o que torna a consulta mais demorada.

O enunciado deste trabalho permitia interpretar o tema de forma aberta e indicava que deviam ser incluídas tantas funcionalidades quanto fosse possível. Por esse motivo, optei por implementar um núcleo simples e estável, em vez de acrescentar funcionalidades que não conseguiria testar ou explicar corretamente. A aplicação foi desenvolvida no terminal, sem interface gráfica, para manter o foco na lógica do programa e na ligação à base de dados.

O projeto também serviu para aplicar conteúdos abordados na unidade curricular, nomeadamente variáveis, condições, ciclos, listas, funções, módulos, exceções, bibliotecas, acesso a bases de dados e escrita de ficheiros. A utilização do MySQL permitiu que os dados permanecessem guardados entre execuções do programa.

No final, o AeroGest ficou preparado para ser colocado num repositório GitHub, incluindo apenas os ficheiros necessários e excluindo configurações locais, ambientes virtuais e credenciais.

3 - DESCRIÇÃO DO PROBLEMA

O problema proposto consistia em criar uma aplicação de gestão de aeródromo. Como o tema não definia uma lista fechada de entidades, foi necessário escolher um âmbito realista. Foram selecionadas duas áreas principais: a gestão das aeronaves e o registo dos seus movimentos.

Cada aeronave possui um identificador, uma matrícula, um modelo, uma capacidade e um estado. A matrícula é única para evitar registos duplicados. Os estados usados na aplicação são Disponível, Em voo e Manutenção. Esta informação permite consultar rapidamente a situação atual de cada aeronave.

Os movimentos representam chegadas ou partidas. Cada movimento fica associado a uma aeronave através do seu identificador e inclui o tipo, a origem ou destino e a data e hora. Esta relação permite mostrar a matrícula da aeronave sempre que os movimentos são consultados.

Além do registo e consulta dos dados, o programa precisava de impedir entradas incorretas, apresentar estatísticas simples e criar um relatório que pudesse ser aberto no navegador. A aplicação tinha também de funcionar após ser fechada e aberta novamente, sem perder os dados já inseridos.

4 - DESENVOLVIMENTO

4.1 - Estrutura da aplicação

O projeto foi dividido em três ficheiros Python principais. O ficheiro `main.py` contém o menu, a leitura dos dados e as funções usadas pelo utilizador. O ficheiro `database.py` contém a ligação ao MySQL e as consultas SQL. O ficheiro `relatorio.py` obtém os dados e escreve o relatório HTML.

O ficheiro `db_config.exemplo.py` serve como exemplo de configuração. A cópia local, chamada `db_config.py`, contém a palavra-passe do MySQL e não é enviada para o GitHub. O ficheiro `dependencias.txt` indica a biblioteca `mysql-connector-python`, necessária para a comunicação entre Python e MySQL.

4.2 - Menu principal

O programa começa por inicializar a base de dados e depois entra num ciclo que apresenta o menu. A opção introduzida pelo utilizador é comparada através de condições `if` e `elif`. Cada opção chama uma função diferente e a opção zero termina o ciclo.

```
=====
                        AEROGEST
                Gestão simples de aeródromo
=====

1. Adicionar aeronave
2. Listar aeronaves
3. Alterar estado da aeronave
4. Registar chegada ou partida
5. Listar movimentos
6. Filtrar movimentos
7. Mostrar estatísticas
8. Gerar relatório HTML

0. Sair

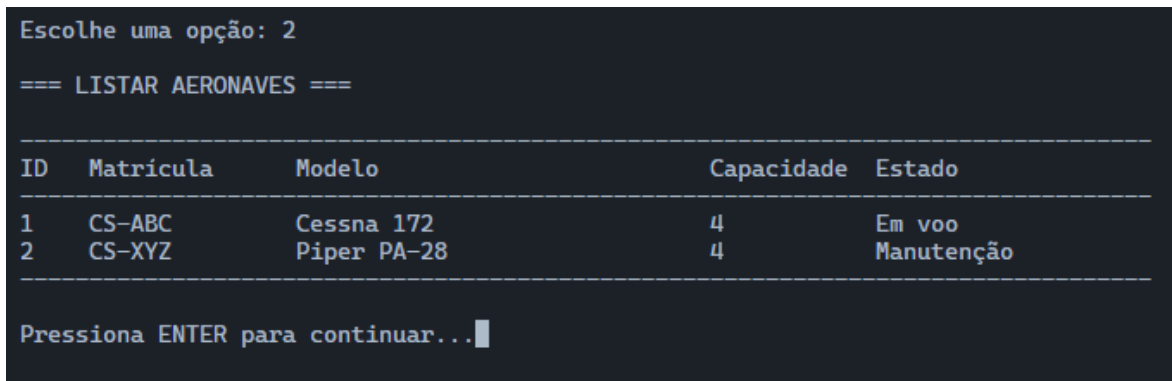
Escolhe uma opção: █
```

Figura 1 - Menu principal da aplicação AeroGest.

4.3 - Gestão de aeronaves

A primeira área da aplicação permite adicionar aeronaves e consultar os registos existentes. Para adicionar uma aeronave são pedidos a matrícula, o modelo, a capacidade e o estado. A matrícula é convertida para maiúsculas e a capacidade tem de ser um número inteiro superior a zero.

A listagem apresenta os dados alinhados no terminal. Também existe uma opção para alterar apenas o estado de uma aeronave, usando o seu identificador. Esta atualização é confirmada na base de dados através do método commit.



```
Escolhe uma opção: 2
=== LISTAR AERONAVES ===
```

ID	Matrícula	Modelo	Capacidade	Estado
1	CS-ABC	Cessna 172	4	Em voo
2	CS-XYZ	Piper PA-28	4	Manutenção

```
Pressiona ENTER para continuar...
```

Figura 2 - Aeronaves registadas na base de dados.

Na demonstração foram registadas duas aeronaves. A CS-ABC, modelo Cessna 172, ficou com o estado Em voo. A CS-XYZ, modelo Piper PA-28, ficou em Manutenção. Estes registos foram usados nos restantes testes do programa.

4.4 - Gestão de movimentos

A opção de registo de movimentos começa por mostrar as aeronaves disponíveis. O utilizador escolhe o identificador da aeronave, indica se o movimento é uma chegada ou uma partida, introduz a origem ou destino e escreve a data e hora no formato definido pelo programa.

Antes de guardar o movimento, o programa confirma se a aeronave existe. Na consulta dos movimentos é utilizado um JOIN entre as tabelas aeronaves e movimentos. Desta forma, a listagem apresenta a matrícula em vez de mostrar apenas o identificador interno.

```
=====
          AEROGEST
    Gestão simples de aeródromo
=====

1. Adicionar aeronave
2. Listar aeronaves
3. Alterar estado da aeronave
4. Registar chegada ou partida
5. Listar movimentos
6. Filtrar movimentos
7. Mostrar estatísticas
8. Gerar relatório HTML

0. Sair

Escolhe uma opção: 5

=== LISTAR MOVIMENTOS ===

-----
ID   Matrícula   Tipo      Origem/Destino   Data/Hora
-----
1    CS-ABC      Partida   Lisboa           2026-07-15 09:30
2    CS-ABC      Chegada   Porto            2026-07-15 13:00
-----

Pressiona ENTER para continuar...
```

Figura 3 - Movimentos de chegada e partida registados.

Para os testes foi registada uma partida para Lisboa às 09:30 e uma chegada proveniente do Porto às 13:00. Os dois movimentos ficaram associados à aeronave CS-ABC.

4.5 - Filtragem de movimentos

A opção de filtragem permite escolher Chegada ou Partida. O tipo selecionado é enviado como parâmetro para a função de consulta, que acrescenta uma condição WHERE à instrução SQL. Assim, a mesma função pode apresentar todos os movimentos ou apenas os movimentos do tipo pretendido.

```
Escolhe uma opção: 6

=== FILTRAR MOVIMENTOS ===

Escolhe o tipo
1. Chegada
2. Partida
Escolha: 1

-----
ID   Matrícula   Tipo      Origem/Destino      Data/Hora
-----
2    CS-ABC      Chegada   Porto                2026-07-15 13:00
-----

Pressiona ENTER para continuar...
```

Figura 4 - Filtro aplicado para mostrar apenas chegadas.

```
Escolhe uma opção: 6

=== FILTRAR MOVIMENTOS ===

Escolhe o tipo
1. Chegada
2. Partida
Escolha: 2

-----
ID   Matrícula   Tipo      Origem/Destino      Data/Hora
-----
1    CS-ABC      Partida   Lisboa               2026-07-15 09:30
-----

Pressiona ENTER para continuar...
```

Figura 5 - Filtro aplicado para mostrar apenas partidas.

4.6 - Validação e tratamento de erros

Foram criadas funções de apoio para validar os valores introduzidos no terminal. Os campos de texto não podem ficar vazios, os identificadores e capacidades têm de ser números inteiros positivos e as opções têm de pertencer às listas apresentadas. A data e hora é verificada com a biblioteca datetime.

A matrícula da aeronave foi definida como única na base de dados. Quando o utilizador tenta registar novamente a mesma matrícula, o erro é tratado e é apresentada uma mensagem simples, sem terminar inesperadamente a aplicação.

```
=====
                AEROGEST
          Gestão simples de aeródromo
=====

1. Adicionar aeronave
2. Listar aeronaves
3. Alterar estado da aeronave
4. Registar chegada ou partida
5. Listar movimentos
6. Filtrar movimentos
7. Mostrar estatísticas
8. Gerar relatório HTML

0. Sair

Escolhe uma opção: 1

=== ADICIONAR AERONAVE ===
Matricula:
O campo não pode ficar vazio.
Matricula: CS-ABC
Modelo: Cessna 172
Capacidade: abc
Introduz um número inteiro válido.
Capacidade: 0
O número deve ser superior a zero.
Capacidade: 4

Estado da aeronave
1. Disponível
2. Em voo
3. Manutenção
Escolha: 9
Escolha uma opção válida.
Escolha: 1

Ocorreu um erro: Já existe uma aeronave com essa matrícula.

Pressiona ENTER para continuar...|
```

Figura 6 - Exemplos de validação e tratamento de dados inválidos.

4.7 - Base de dados

A aplicação utiliza o MySQL para guardar os dados. Na primeira execução, a função de inicialização cria a base aerogest, caso ainda não exista, e cria as tabelas aeronaves e movimentos. Esta preparação automática evita que o utilizador tenha de criar manualmente as tabelas antes de executar o programa.

A tabela aeronaves guarda o identificador, a matrícula, o modelo, a capacidade e o estado. O identificador é criado automaticamente e a matrícula possui uma restrição de unicidade. A tabela movimentos guarda o identificador, a aeronave associada, o tipo, a origem ou destino e a data e hora.

A relação entre as tabelas é feita pelo campo `aeronave_id`. Este campo é uma chave estrangeira que aponta para o identificador da tabela aeronaves. Desta maneira, um movimento não fica isolado e pode ser relacionado com a aeronave correta.

As inserções são realizadas com `INSERT`, as listagens com `SELECT` e a alteração do estado com `UPDATE`. Os valores não são escritos diretamente dentro das instruções SQL. São enviados separadamente através de parâmetros, o que também evita problemas com caracteres introduzidos pelo utilizador.

Depois de uma inserção ou alteração é usado `commit` para confirmar a operação. As ligações e os cursores são fechados no bloco `finally`, mesmo quando ocorre um erro. Esta organização reduz a possibilidade de ficarem ligações abertas sem necessidade.

4.8 - Geração do relatório HTML

A opção oito cria um relatório em formato HTML. A função começa por consultar as aeronaves, os movimentos e as estatísticas. Em seguida, percorre os resultados e constrói as linhas das duas grelhas apresentadas no navegador.

O ficheiro é guardado numa pasta chamada relatorios e recebe no nome a data e a hora em que foi criado. Isto evita que um relatório anterior seja substituído automaticamente. O conteúdo inclui os totais, a listagem das aeronaves e a listagem dos movimentos.

Relatório AeroGest

Gerado em 12/07/2026 às 22:28

Estatísticas

Total de aeronaves: 2

Aeronaves em manutenção: 1

Chegadas: 1

Partidas: 1

Aeronaves

ID	Matricula	Modelo	Capacidade	Estado
1	CS-ABC	Cessna 172	4	Em voo
2	CS-XYZ	Piper PA-28	4	Manutenção

Movimentos

ID	Matricula	Tipo	Origem/Destino	Data/Hora
1	CS-ABC	Partida	Lisboa	2026-07-15 09:30
2	CS-ABC	Chegada	Porto	2026-07-15 13:00

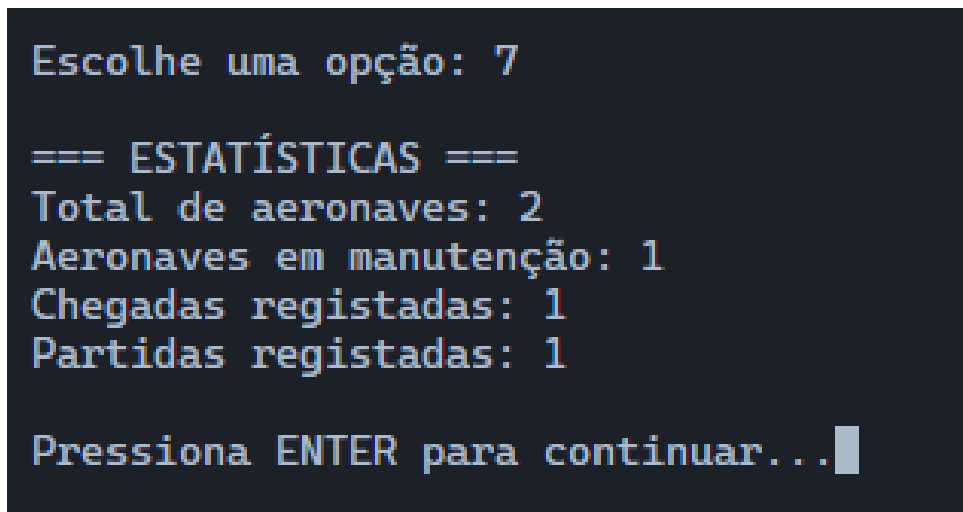
Figura 7 - Relatório HTML final gerado pela aplicação.

O relatório mostrado na figura foi gerado depois de corrigir e confirmar os dados finais. As informações apresentadas correspondem aos valores existentes no MySQL no momento da geração.

5 - RESULTADOS

5.1 - Resultados funcionais

No final dos testes, a aplicação apresentava duas aeronaves, uma em manutenção, uma chegada e uma partida. Estes valores foram calculados diretamente com consultas COUNT ao MySQL e coincidiram com os registos mostrados nas listagens.

A terminal window with a dark background and light-colored text. The text displayed is: 'Escolhe uma opção: 7', followed by '=== ESTATÍSTICAS ===', then 'Total de aeronaves: 2', 'Aeronaves em manutenção: 1', 'Chegadas registadas: 1', and 'Partidas registadas: 1'. At the bottom, it says 'Pressiona ENTER para continuar...' with a cursor icon at the end.

```
Escolhe uma opção: 7

=== ESTATÍSTICAS ===
Total de aeronaves: 2
Aeronaves em manutenção: 1
Chegadas registadas: 1
Partidas registadas: 1

Pressiona ENTER para continuar...
```

Figura 8 - Estatísticas calculadas a partir dos dados guardados.

Também foi confirmada a persistência. Depois de escolher a opção de saída e executar novamente o programa, as duas aeronaves e os dois movimentos continuaram disponíveis. Este teste demonstrou que os dados não estavam guardados apenas em variáveis durante a execução.

O relatório HTML foi criado sem erros e mostrou os mesmos dados do terminal. A base de dados ficou com os registos usados na demonstração e o projeto foi preparado para publicação num repositório Git, sem incluir a palavra-passe ou o ambiente virtual.

5.2 - Testes realizados

Os testes funcionais abrangeram a criação de aeronaves, a listagem dos dados, a alteração do estado, o registo de uma partida e de uma chegada, a listagem dos movimentos, os dois filtros, as estatísticas, a geração do relatório e a persistência após reiniciar o programa.

Foram também testados vários erros de introdução. O programa rejeitou um campo vazio, uma capacidade escrita com letras, uma capacidade igual a zero, uma opção inexistente, uma matrícula repetida, um identificador que não existia e uma data escrita no formato errado. Em cada caso foi apresentada uma mensagem e o programa permitiu continuar.

5.3 - Problemas encontrados e resolução

A primeira dificuldade surgiu na ligação ao MySQL. A biblioteca `mysql-connector-python` estava instalada, mas não existia um servidor MySQL ativo na porta 3306. O teste de ligação devolveu o erro 2003, indicando que não era possível estabelecer ligação a localhost.

Para resolver o problema foi instalado e configurado o MySQL Server. Foi criada uma palavra-passe para o utilizador root, preenchido o ficheiro local `db_config.py` e testada novamente a ligação. Depois dessa configuração, o programa conseguiu criar a base de dados e arrancou normalmente.

Durante os testes foi também detetado que o modelo da aeronave CS-ABC tinha sido introduzido apenas como 172. O registo foi corrigido para Cessna 172 antes de gerar as capturas finais. Os movimentos temporários usados nos testes de validação foram removidos para que o relatório final apresentasse apenas os dados da demonstração.

6 - CONCLUSÕES

O desenvolvimento do AeroGest permitiu criar uma aplicação funcional para uma gestão básica de aeródromo. O programa regista aeronaves, altera o seu estado, guarda chegadas e partidas, apresenta listagens, aplica filtros, calcula estatísticas e gera um relatório que pode ser consultado no navegador.

A opção por uma aplicação de terminal tornou o projeto mais simples de testar e compreender. Mesmo sem uma interface gráfica, foi possível aplicar os conteúdos principais da unidade curricular e estabelecer uma ligação real entre Python e uma base de dados.

Os testes mostraram que as funcionalidades implementadas funcionam e que os erros mais comuns são tratados sem fechar a aplicação. A persistência dos dados também foi confirmada. O projeto ficou limitado a duas entidades para manter um âmbito realista, mas a estrutura permite acrescentar outras áreas no futuro.

Como melhorias futuras poderiam ser adicionadas a gestão de pilotos, pistas e funcionários, a edição completa dos registos, a eliminação controlada, mais filtros e uma interface gráfica. Estas possibilidades não foram incluídas nesta versão para evitar aumentar a complexidade sem tempo suficiente para testar e explicar todas as alterações.

7 - REFERÊNCIAS

Material pedagógico das aulas da unidade curricular de Programação Avançada, disponibilizado pelo docente durante o ano letivo 2025/2026.

Python Software Foundation. Documentação da linguagem Python 3, consultada para funções, exceções, módulos, datetime e escrita de ficheiros.

Oracle. Documentação do MySQL e do MySQL Connector/Python, consultada para a ligação à base de dados e execução de instruções SQL.

Código e capturas de ecrã produzidos durante a implementação e os testes do projeto AeroGest.